

Du wirst



VHDL lieben!

Auszug Lehrplan

Die Schülerinnen und Schüler können im III. Jahrgang, 6. Semester im Bereich Entwurf digitaler Systeme

Schaltwerke entwerfen und in programmierbaren Logikbausteinen implementieren.

Dieses Skript wurde von **Michael Kupfner** für das Unterrichtsfach **DIC – Digitale Systeme und Computersysteme, 4aHEL, Schuljahr 2025/26** an der HTBLVA Anichstraße erstellt. Eine Weitergabe oder Vervielfältigung außerhalb dieses Unterrichts ist nicht gestattet.

1 Prozesse

Bisher wurden alle beschriebenen Schaltungen **nebenläufig** (d.h. parallel) umgesetzt. Für viele Entwürfe reicht dies aus – aber sobald wir **Schaltwerke (Automaten)** beschreiben wollen, benötigen wir **sequentielle Abläufe**.

In VHDL können **Prozesse** beschrieben werden. Innerhalb eines Prozesses erfolgt die Abarbeitung **sequentiell**, ähnlich wie in einer klassischen Programmiersprache. Damit stehen uns auch Verzweigungen (*if, case*) und Schleifen (*for, while*) zur Verfügung.

Ein Prozess beginnt mit dem VHDL Schlüsselwort **PROCESS**. Gibt es in einer **ARCHITECTURE** mehrere Prozesse, so werden diese wiederum nebenläufig (d.h. parallel) abgearbeitet. Die Kommunikation zwischen Prozesse erfolgt durch Signale.

1.1 Allgemeine Form eines Prozesses

Deklaration eines Prozesses:

```
[<Prozessname>:] process [(<Sensitivitätsliste>)]  
    -- Deklarationen (Variablen, Konstanten)  
begin  
    -- sequentielle Anweisungen  
end process [<Prozessname>;]
```

- **Prozessname:** optional, erleichtert aber die Fehlersuche.
- **Sensitivitätsliste:** enthält Signale, deren Änderung den Prozess neu startet.
- **Deklarationen:** lokale Variablen oder Konstanten, nur innerhalb des Prozesses gültig.
- **Sequentielle Anweisungen:** werden beim Start des Prozesses nacheinander abgearbeitet.

Es gibt zwei -sich gegenseitig ausschließende- Möglichkeiten, einen Prozess auszuführen:

1. **Mit Sensitivitätsliste:** Der Prozess startet bei Modell-Initialisierung und wird danach immer dann ausgeführt, wenn sich ein Signal aus der Liste ändert.
2. **Mit WAIT-Anweisung:** Der Prozess läuft bis zum ersten **WAIT** durch und pausiert so lange, bis die Bedingung erfüllt ist.

Prozesse ohne *Sensitivitätsliste* und ohne **WAIT** führen zu einer **Endlosschleife** und sind üblicherweise nicht sinnvoll.

1.2 Prozesse für Schaltnetze und Schaltwerke

Mit Prozesse lassen sich aber nicht nur (takt)synchrone Schaltwerke realisieren, sondern auch Schaltnetze. Enthält die Sensitivitätsliste alle Eingangssignale, dann wird ein **kombinatorisches Schaltnetz** erzeugt.

Dieses Verhalten lässt sich im folgenden Beispiel beobachten. Ein 4-bit Codewort soll mit einem zusätzlichen **Paritätsbit** gesichert werden. Dieses Paritätsbit muss '1' sein, wenn das Codewort eine ungerade Anzahl von Einsen aufweist, ansonsten '0' :

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Parity4 is
  port (
    D : in  std_ulogic_vector(3 downto 0); -- 4 Eingänge
    P : out std_ulogic                    -- Paritätsbit (gerade Parität)
  );
end Parity4;

architecture Verhalten of Parity4 is
begin
  PAR_PROC: process(D)
    variable Par : std_ulogic;
  begin
    Par := '0';           -- Startwert
    Par := Par xor D(0);   -- sequentielle Schritte
    Par := Par xor D(1);
    Par := Par xor D(2);
    Par := Par xor D(3);
    P    <= Par;           -- Ergebnis zuweisen
  end process PAR_PROC;
end Verhalten;

```



Aufgabe: Lege in Quartus ein Projekt mit dieser ENTITY an. Überprüfe anschließend im RTL-Viewer, dass wirklich nur eine kombinatorische Schaltung ohne Flip-Flops erzeugt wird. Führe danach eine Simulation durch und kontrolliere die korrekte Funktion anhand der Wahrheitstabelle.



KI-Aufgabe: In dieser VHDL-Beschreibung wurde erstmals eine **Variable** eingesetzt. Frage eine KI nach dem Unterschied zwischen einem Signal und einer Variablen in VHDL und notiere die beiden wichtigsten Unterschiede in eigenen Worten. Recherchiere außerdem, in welchen Situationen Variablen sinnvoll sind – und beantworte, warum ein Signal an dieser Stelle zu einem anderen Verhalten führt.

Ein **synchrones Schaltwerk** wie z. B. ein Flip-Flop oder ein Register entsteht, wenn in der Sensitivitätsliste typischerweise nur das Taktsignal und gegebenenfalls ein Reset-Signal stehen. Innerhalb des Prozesses wird dann die jeweilige Taktflanke abgefragt.

Das folgende Beispiel zeigt ein **D-Flip-Flop mit asynchronem Reset**.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity D_FF is
  port (
    CLK, RST : in  std_ulogic;
    D         : in  std_ulogic;
    Q         : out std_ulogic
  );
end D_FF;

```

```

architecture Verhalten of D_FF is
begin
  PROC: process(CLK, RST)
  begin
    if (RST = '1') then          -- Reset hat Vorrang
      Q <= '0';
    elsif rising_edge(CLK) then
      Q <= D;
    end if;
  end process PROC;
end Verhalten;

```

Der Prozess *PROC* wird dabei einmal bei der Initialisierung abgearbeitet und danach immer dann neu gestartet, wenn sich *RST* oder *CLK* ändern – also genau die Signale, die in der Sensitivitätsliste stehen.

Sobald *RST*='1' ist, wird der Ausgang *Q* unabhängig vom Takt auf '0' gesetzt (→ asynchroner Reset). Bei *RST*='0' übernimmt *Q* bei jeder *steigenden Taktflanke* den aktuellen Wert von *D*. Damit wird der Eingangswert gespeichert, bis bei der nächsten Taktflanke ein neuer übernommen wird.

i Info: Historische Schreibweise von Taktflanken

Früher wurde eine steigende Taktflanke oft mit *if CLK = '1' and CLK'event then ...* abgefragt (bzw. mit *CLK = '0'* auf fallende Flanke).

Das Attribut *'event'* (ausgesprochen: Tick-Event) zeigt an, dass ein Signal seinen Wert geändert hat.

Problem: Bei Signalen vom Typ *std_logic* / *std_ulogic* wird damit auch ein Übergang von z.B. 'X' nach '1' als Flanke erkannt – das ist oft unerwünscht. Daher wurde in *std_logic_1164* die Funktion **rising_edge** und **falling_edge** eingeführt:

if rising_edge(CLK) then ...

Diese Schreibweise ist präziser und heute der empfohlene Standard.

 **Aufgabe:** Erweitere das D-Flip-Flop um einen **Enable-Eingang**. Der Ausgang *Q* soll den Wert von *D* nur dann übernehmen, wenn Enable *E* aktiv ist. Überprüfe die Funktion per Simulation.

Neben der Sensitivitätsliste kann ein Prozess auch durch **WAIT-Anweisungen** gesteuert werden. Dabei wird der Prozess nach dem Schlüsselwort **BEGIN** zunächst abgearbeitet, bis er auf ein **WAIT** stößt. Danach hält er an, bis die Bedingung erfüllt ist, und setzt seine Arbeit dann fort.

Der folgende VHDL-Codeausschnitt demonstriert dies anhand der Beschreibung eines D-Flip-Flop:

```

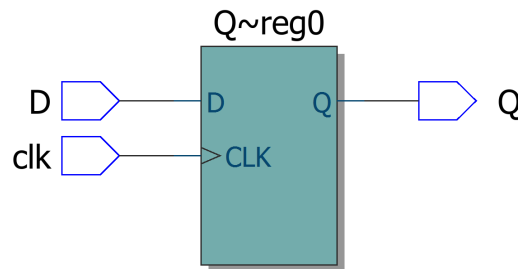
architecture Verhalten of D_FF_wait is
begin
  PROC_D_FF: process
  begin
    wait until rising_edge(CLK); -- Prozess pausiert bis zur nächsten
                                steigenden Taktflanke

    Q <= D;
  end process PROC_D_FF;
end Verhalten;

```

Erklärung: Der Prozess startet bei der Initialisierung und läuft bis zum ersten *wait until rising_edge(CLK)*. Dann pausiert er und setzt erst bei der nächsten steigenden Taktflanke fort. An dieser Stelle wird der Wert von *D* vorgemerkt und nach Abschluss des Prozesses dem Ausgang *Q* zugewiesen. Anschließend beginnt der Prozess wieder von vorn und wartet auf die nächste Taktflanke.

Darstellung RTL-Viewer:



Zusammengefasst:

VHDL Synthese von taktsynchronen Schaltwerken:

- Mit Sensitivitätsliste: Die Liste enthält ausschließlich das Taktsignal (und ggf. Reset). Innerhalb des Prozesses wird die gewünschte Taktflanke mit einer if-Abfrage überprüft. Weitere Abfragen auf Takt oder Taktflanken dürfen innerhalb dieses Prozesses nicht erfolgen.
- Ohne Sensitivitätsliste: Hier wird die Taktflanke mit einer WAIT UNTIL-Anweisung abgefragt. Als guter Beschreibungsstil (und nach IEEE-Empfehlung) soll diese Abfrage genau einmal erfolgen, direkt nach BEGIN. Danach sind keine weiteren Taktabfragen mehr zulässig.

VHDL Synthese von Schaltnetzen:

- Mit Sensitivitätsliste: Alle Eingangssignale, die rechts von Zuweisungen oder in Abfragen (if, case) verwendet werden, müssen in die Sensitivitätsliste aufgenommen werden. Werden alle Eingangssignale vollständig abgebildet, so entsteht ein kombinatorisches Schaltnetz ohne Zustandsspeicher.
- Ohne Sensitivitätsliste: Nicht sinnvoll, da dies zu einem endlosen Prozess führen würde.

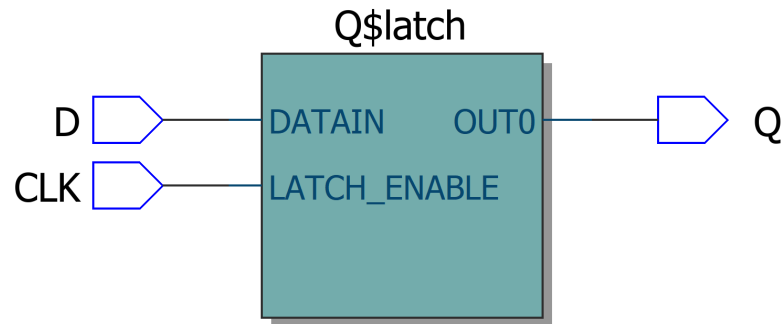
Der folgende VHDL-Codeauszug zeigt die Beschreibung eines D-Latch:

```
PROC_D_LATCH: process(D, CLK)
begin
    if CLK = '1' then
        Q <= D;
    end if;
end process PROC_D_LATCH;
```

Erklärung: In diesem Prozess wird *Q* auf *D* gesetzt, solange *CLK='1'* ist. Sobald *CLK='0'* wird, behält *Q* jedoch seinen letzten Wert. Damit entsteht ein Latch.

Latches werden in FPGAs fast nie bewusst eingesetzt. Sie entstehen oft **versehentlich**, z. B. durch unvollständige IF-Abfragen oder fehlende ELSE-Zweige.

Darstellung RTL-Viewer:



 **KI-Aufgabe:** Frage eine KI, warum Latches in FPGA-Designs problematisch sind. Notiere mindestens zwei Gründe und bewerte, ob diese in deinem eigenen Entwurf relevant wären.

1.3 Variablen und Signale in Prozessen

Innerhalb von Prozessen können sowohl **Variablen** als auch **Signale** verwendet werden. Beide haben eine unterschiedliche zeitliche Wirkung:

Variablen:

- sind lokal und temporär, sie existieren nur während der „Laufzeit“ des Prozesses.
- ihr Wert wird **sofort** bei der Abarbeitung einer Anweisung geändert.
- auf Variablen kann nicht von außerhalb des Prozesses zugegriffen werden.
- beim nächsten Aktivieren des Prozesses besitzen sie wieder den **alten Wert** (den Wert vor Prozessende).
- nicht jede Variable führt bei der Synthese zu einem Hardwareelement – sie können auch „wegoptimiert“ werden.

Signale:

- sind global sichtbare Verbindungen innerhalb einer ARCHITECTURE.
- innerhalb eines Prozesses gilt: Zuweisungen an Signale werden erst **nach Abschluss des Prozesses** wirksam (→ vorgemerkt).
- wenn ein Wert auch außerhalb des Prozesses sichtbar sein soll, muss er einem Signal zugewiesen werden.

 **Merke:** Variablen ändern sich **sofort**, Signale erst **nach Prozessende**.

Anmerkung zur Sensitivitätsliste:

Eine Sensitivitätsliste bewirkt, dass ein Prozess neu gestartet wird, sobald sich eines der darin enthaltenen Signale ändert.

Ein *WAIT ON* am Ende eines Prozesses hat die gleiche Wirkung wie eine Sensitivitätsliste.

Wird ein Signalvektor (Bus) verwendet, genügt bereits die Änderung eines einzelnen Bits, um den Prozess neu zu starten.

1.4 IF-ELSIF-ELSE - Anweisung

Bedingte Verzweigungen in **sequentiellen** Anweisungsteilen (also z. B. innerhalb eines PROCESS) werden mit der **IF-ELSIF-ELSE**-Struktur formuliert:

```
if <Bedingung_1> then
    -- sequentielle Anweisungen

elsif <Bedingung_2> then
    -- sequentielle Anweisungen

elsif <Bedingung_n> then
    -- sequentielle Anweisungen

else
    -- Default-Zweig (optional, aber empfohlen)

end if;
```

 **Merke:** Zwingend erforderlich ist nur der erste **IF**-Zweig und das abschließende **END IF**; **ELSIF** kann beliebig oft wiederholt werden; **ELSE** ist optional, **vermeidet aber Latches** (Default-Zuweisung).

Wichtig: Die Bedingung muss ein **BOOLEAN**-Ausdruck sein; also Signale vom Typ **boolean** oder eine logische Vergleichsaussage (wie z.B. *S1='0'*).

Schlecht (Typfehler + potentielles Latch):

```
architecture if_elsif_else of Demo is
    signal S1 : std_ulogic;
    signal B1 : boolean;
begin
    PROC: process(S1, B1)
    begin
        if S1 then                -- FEHLER: S1 ist kein BOOLEAN
            ...
        elsif B1 and S1 then     -- FEHLER: S1 again kein BOOLEAN
            ...
        end if;                  -- kein else -> Default fehlt (Latch-Gefahr)
    end process PROC;
end if_elsif_else;
```

Korrekt (typsicher, mit Default):

```
architecture if_elsif_else of Demo is
    signal S1 : std_ulogic;
    signal B1 : boolean;
begin
    PROC: process(S1, B1)
    begin
        if S1 = '1' then
            ...
        elsif B1 then
            ...
        else
            ...
        end if;
    end process PROC;
end if_elsif_else;
```



```
    end if;  
    end process PROC;  
end if_elsif_else;
```

Der folgende VHDL-Beschreibungsauszug erzeugt eine sog. „incomplete branch“ und ist auf jeden Fall zu vermeiden:

```
process(A,B)  
begin  
    if (A=B) then  
        X <= '1';           -- beide Eingangssignale sind gleich → Ausgang Log. 1  
    end if;  
end process;
```

Ausgaben während der Kompilierung:

Warning (10631): VHDL Process Statement warning at Incomplete_Branch.vhd(13): inferring latch(es) for signal or variable "X", which holds its previous value in one or more paths through the process

Info (10041): Inferred latch for "X" at Incomplete_Branch.vhd(13)

... da die oberen Codezeilen gleichbedeutend sind mit

```
process(A,B)  
begin  
    if (A=B) then  
        X <= '1';           -- beide Eingangssignale sind gleich → Ausgang Log. 1  
    else  
        X <= X;  
    end if;  
end process;
```

Der Synthesizer inferenziert hier ein **Latch**, da für den Fall ($A \neq B$) kein neuer Wert zugewiesen wird (implizit „ $X \leq X$;“).

Korrekt (vollständig, latchfrei):

```
process(A, B)  
begin  
    if (A = B) then  
        X <= '1';  
    else  
        X <= '0';  
    end if;  
end process;
```

 **Merke:** In kombinatorischen Prozessen gehört **jedes gelesene Signal** in die Sensitivitätsliste, und **jeder Ausgang** muss in **allen** Pfaden einen Wert erhalten (sonst: inferred latch).

 **Aufgabe:** Entwirf einen 4-zu-1 Multiplexer in einem kombinatorischen Prozess mit *IF-ELSIF-ELSE*. Verifiziere im RTL-Viewer, dass **kein Speicherbaustein** eingebunden wird. Überprüfe die korrekte Funktion mittels Simulation.

1.5 Native Datentypen in VHDL

Bisher haben wir in unseren Beispielen überwiegend mit den Typen *bit* und *std_(u)logic* gearbeitet, im Kapitel 1.4 kam auch bereits *boolean* vor. VHDL stellt jedoch eine Reihe von **nativen Datentypen** bereit, die in vielen Situationen hilfreich oder sogar notwendig sind. Dazu gehören insbesondere die Typen *boolean* für logische Ausdrücke sowie *integer* für Zähler und Berechnungen. Einige weitere Typen (z. B. *real*, *character*, *string*) spielen vor allem bei der Simulation (in Testbenches) eine Rolle.

Kurzübersicht:

- BIT / BIT_VECTOR: minimaler, binärer Typ (nur '0' und '1'), guter Einstieg, aber kaum in der Praxis verwendet.
- BOOLEAN: logische Werte TRUE/FALSE, nützlich für Bedingungen (*if*, *while*).
- INTEGER: ganze Zahlen; erlaubt negative und positive Werte. Bereich: meist -2^{31} bis $-2^{31}-1$.
- NATURAL: ganze Zahlen; Subtyp von Integer, erlaubt nur positive Werte inkl. 0. Bereich: 0 bis $+2^{31}-1$.
- REAL: für Simulation (Gleitkomma-Berechnungen in Testbenches), nicht für Synthese.
- CHARACTER, STRING, TIME: für Testbenches (Simulation), nicht für Synthese.

Hinweis: std_ulogic / std_logic sind keine nativen Datentypen, sondern stammen aus der IEEE-1164-Bibliothek (daher „quasi-native“ in der Praxis).

1.6 CASE - Anweisung

Mit *CASE* lassen sich **wertbasierte Auswahlregeln** sehr kompakt formulieren – ideal für **Tabellenlogik** wie Decoder. Im FPGA-Kontext ist das nicht nur übersichtlich, sondern auch effizient: Kleine Wertetabellen landen praktisch 1:1 in SRAM-LUTs. Im Gegensatz zu *IF-ELSIF-ELSE* erzeugt *CASE* **keine Prioritätslogik** (die Zweige sind wechselseitig exklusiv).

Beispiel: BCD → 7-Segment-Decoder (DE0: low-aktiv)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Segmentdecoder is
    port ( bcd:      in std_ulogic_vector(3 downto 0);
          sseg:      out std_ulogic_vector(6 downto 0)  -- g f e d c b a
        );
end Segmentdecoder;

architecture case_implementation of Segmentdecoder is
begin
    DEC:process(bcd)
        variable temp_sseg : std_ulogic_vector(6 downto 0); -- gfedcba, aktiv-HIGH
        begin
            case bcd is
                -- 7-Segment-Ansteuerung: g f e d c b a (aktiv-HIGH)
                when "0000" => temp_sseg := "0111111"; -- 0
                when "0001" => temp_sseg := "0000110"; -- 1
                when "0010" => temp_sseg := "1011011"; -- 2
                when "0011" => temp_sseg := "1001111"; -- 3
                when "0100" => temp_sseg := "1100110"; -- 4
```

```

    when "0101" => temp_sseg := "1101101"; -- 5
    when "0110" => temp_sseg := "1111101"; -- 6
    when "0111" => temp_sseg := "0100111"; -- 7
    when "1000" => temp_sseg := "1111111"; -- 8
    when "1001" => temp_sseg := "1101111"; -- 9
    when others => temp_sseg := "0000000"; -- Anzeige AUS
end case;

-- DE0-Board Segmente Low-aktiv -> daher temp_sseg invertieren
sseg <= not temp_sseg;

end process DEC;
end case_implementation;

```

Beachte...

- Sensitivitätsliste nur *bcd* → kombinatorischer Prozess (kein FF).
- Vektor-Reihenfolge klar dokumentieren (*g f e d c b a*), das vermeidet Vertauschungen.
- Default-Fall (*others*) deckt auch Meta-Zustände ab ('U', 'X', ...) **und verhindert Latches**.

 **Merke:** CASE erfordert eine vollständige Abdeckung aller Fälle. Für **std_(u)logic_vector** ist **WHEN OTHERS** praktisch immer nötig (außer du listest wirklich alle 2^n Kombinationen).

 **Aufgabe:** Erstelle eine neue VHDL-Beschreibung eines BCD-7-Segment-Dekoders, in der statt der Variable *temp_sseg* ein internes **Signal** verwendet wird. Was ändert sich **syntaktisch** und **im Simulationszeitpunkt** der Zuweisung?

 **Aufgabe:** Erweitere den 7-Segment-Decoder um die **hexadezimale Ausgabe A–F**.
Anmerkung: Der Eingangsvektor ist konsequenterweise auf *hex* zu ändern (anstelle von *bcd* – warum?).

KI-Aufgabe: Gray-Code-Umsetzer

Erkläre mit Hilfe einer KI, was ein **Gray-Code** ist. Beschreibe in eigenen Worten, warum sich bei jeder Zählstufe nur ein Bit ändert und in welchen Anwendungen dieser Vorteil genutzt wird. Erstelle anschließend eine VHDL-Beschreibung für einen Umsetzer, der einen **4-Bit-Gray-Code in einen 4-Bit-Binär-Code** wandelt. Verwende dazu die **CASE**-Anweisung.

1.7 Schleifen

VHDL unterstützt – wie viele sequentielle Sprachen – **Schleifen**. Da Schleifen **sequentiell** abgearbeitet werden, dürfen sie nur in sequentiellen Bereichen stehen: insbesondere in PROCESS sowie in Funktionen/Prozeduren. (→ Teil 3; Für nebenläufige Wiederholung existiert separat *generate... wird ebenfalls später behandelt*).

VHDL kennt dabei die folgenden 3 Schleifen:

```

[Label:] for <index> in <low> to <high> loop
    -- sequentielle Anweisungen
end loop [Label];

```

```
[Label:] while <Bedingung> loop
    ...
end loop [Label];
```

```
[Label:] loop
    ...
    exit when <Bedingung>;
end loop [Label];
```

Der **FOR-Index** muss **explizit** als Konstante deklariert werden (nicht zuweisen!). Für synthetisierbaren Code sind **FOR**-Schleifen mit statischer Iterationszahl Standard; **WHILE** wird primär in Testbenches (Simulation) verwendet.

Beispiel für eine FOR-Schleife - Paritätsprüfung (gerade Parität) – generisch & richtungsrobust:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

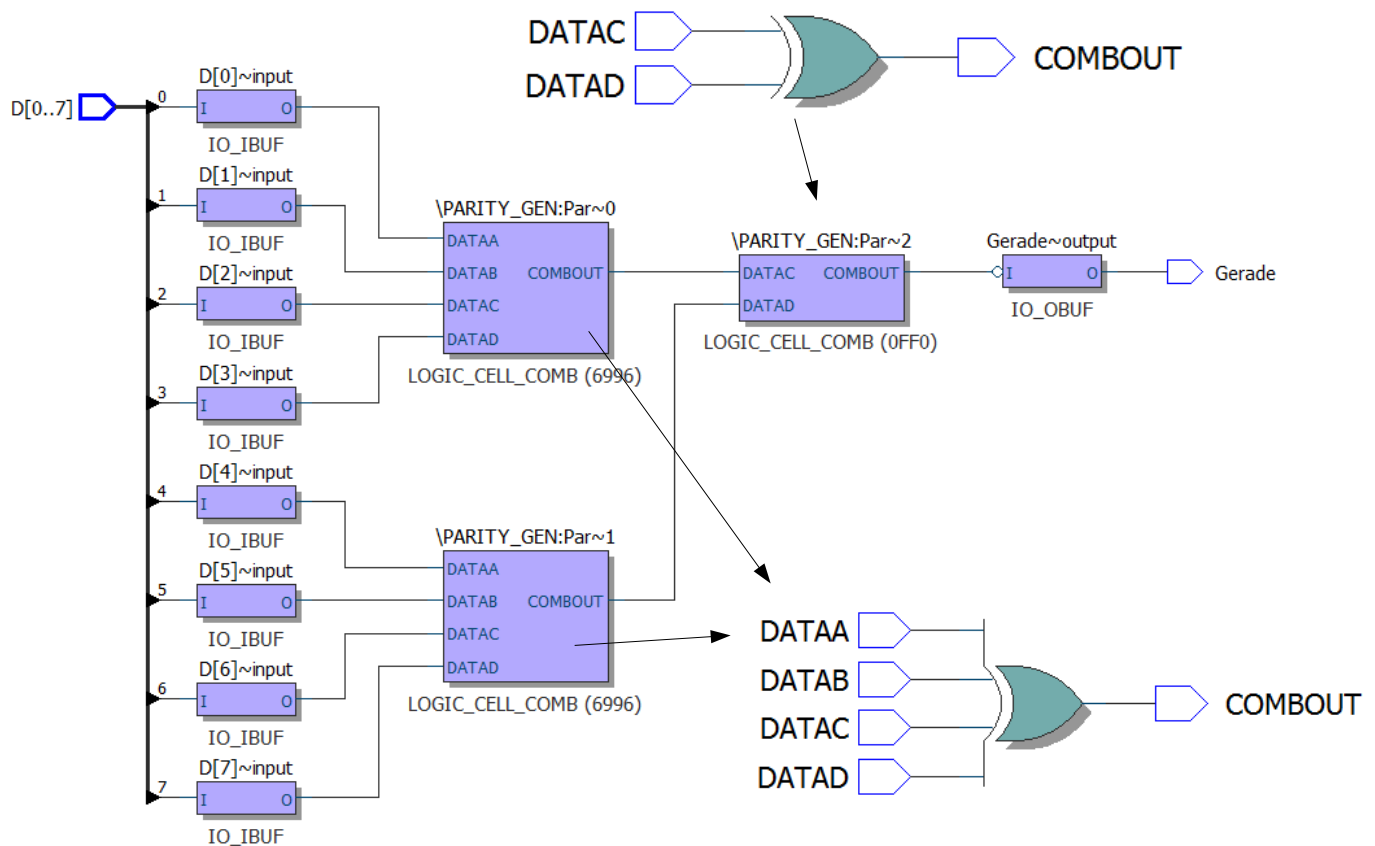
entity Parity_Checker is
    port (
        D      : in  std_ulogic_vector(7 downto 0);
        Gerade : out std_ulogic
    );
end Parity_Checker;

architecture Verhalten of Parity_Checker is
begin
    PARITY_GEN: process(D)
        variable odd : boolean; -- true = ungerade Anzahl '1'
    begin
        odd := false;
        -- richtungsrobust: Läuft über D'range, egal ob downto/to
        for i in D'range loop
            odd := odd xor (D(i) = '1');
        end loop;

        if odd then
            Gerade <= '0'; -- ungerade Anzahl von '1'
        else
            Gerade <= '1'; -- gerade Anzahl von '1'
        end if;
    end process PARITY_GEN;
end Verhalten;
```

Im Beispiel wird ein 8-Bit-Datenvektor überprüft. Die Variable *odd* wird zu Beginn auf *false* gesetzt und in der **FOR**-Schleife bei jedem '1'-Bit negiert. Am Ende signalisiert der Ausgang *Gerade*, ob die Anzahl der Einsen gerade ist (*Gerade* = '1'), andernfalls ungerade.

Synthese-Ergebnis (Tools → Netlist Viewers → Technology Map Viewer (Post Mapping)):



Aufgabe: Bit-Invertierung mit FOR-SCHLEIFE

Erstelle eine VHDL-Beschreibung, die jedes Bit eines 8-Bit-Vektors D invertiert. Verwende dazu eine **FOR-Schleife** (und nicht die Kurzschreibweise `not D`). Überprüfe die korrekte Funktion in einer Simulation.

Aufgabe: Bit-Reverse

Schreibe eine VHDL-Beschreibung, die die Bitreihenfolge eines Vektors umkehrt. Die ENTITY besitzt den Eingang D (`std_ulogic_vector(7 downto 0)`) sowie den Ausgang R (`std_ulogic_vector(7 downto 0)`).

Liegt am Eingang D beispielsweise das Muster "10110000", so soll am Ausgang das Muster "00001101" erscheinen.

Verwende eine FOR-Schleife, um jedes Bit von D an die umgekehrte Position in R zu kopieren. Simuliere dein Design und überprüfe, dass alle Bitmuster korrekt gespiegelt werden.

2 Rechnen in VHDL

VHDL ist eine streng typisierte Sprache. Das bedeutet: Für jedes Objekt (Signal, Variable, Konstante) muss genau festgelegt sein, **welche Werte es annehmen darf**. Der Vorteil: Viele Fehler werden schon beim Kompilieren erkannt.

Beispiel:

```
variable X : integer range 0 to 10;  
X := X + 1;
```

Hier ist klar: *X* darf nur Werte zwischen 0 und 10 annehmen. Ein Überlauf würde vom Compiler oder Simulator gemeldet werden.

2.1 Native Zahlentypen

- INTEGER: Ganze Zahlen (Bereich Tool-abhängig, meist $\pm 2^{31}-1$). Für Synthese sollte der Wertebereich durch **RANGE** eingeschränkt werden.
- NATURAL: Subtyp von INTEGER, nur ≥ 0 .
- REAL: Gleitkommazahlen – nur für Simulation, nicht synthesefähig.

2.2 Rechnen mit std_logic-Vektoren

In der Praxis arbeiten wir meist mit Vektoren (z. B. Bus). Um damit rechnen zu können, wird das Paket *numeric_std* verwendet. Dieses definiert die Typen:

- **UNSIGNED**: Vektor interpretiert als positive Zahl
- **SIGNED**: Vektor interpretiert mit Vorzeichen (2er-Komplement)

Beispiel:

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use IEEE.NUMERIC_STD.ALL;  
  
signal S : unsigned(7 downto 0); -- 0 bis 255  
signal C : signed(7 downto 0);   -- -128 bis +127
```

2.3 Operatoren

Mit *numeric_std* stehen alle Grundoperationen zur Verfügung:

- Addition, Subtraktion: +, -
- Multiplikation: * (über DSP-Blöcke)
- Vergleich: =, /=, <, <=, >, >=
- Division und Modulo: eingeschränkt unterstützt, meist nur für Potenzen von 2 sinnvoll

Multiplikationen sind synthesesfähig. Divisionen sind kritisch: oft nur über IP-Cores oder Speziallogik möglich.

2.4 Typumwandlungen

Da VHDL eine stark typisierende Sprache ist, werden *std_logic_vector*, *unsigned* und *signed* als unterschiedliche Datentypen behandelt. Das mag überraschen, bedenkt man, dass alle diese Datentypen auf Arrays von Elementen des Typs *std_logic* basieren. Man benötigt daher auch Typ-Umwandlungsfunktionen.

Die folgende Tabelle gibt eine Zusammenfassung dieser Funktionen:

Ausgangsdatentyp von a	Ziel-Datentyp	Konvertierungsfunktion
unsigned, signed	std_ulogic_vector	std_ulogic_vector(a)
signed, std_ulogic_vector	unsigned	unsigned(a)
unsigned, std_ulogic_vector	signed	signed(a)
unsigned, signed	Integer	to_integer(a)
integer, natural	unsigned	to_unsigned(a, Länge)
integer, natural	signed	to_signed(a, Länge)

Bei der Konvertierung einer Ganzzahl ist zusätzlich die gewünschte **Vektor-Länge** anzugeben.

Beispiel:

```
X <= to_unsigned(123, X'length);
Y <= to_signed(123, Y'length);
```

Der zweite Parameter legt fest, wie viele Bits für die Darstellung verwendet werden. Dies kann größer sein als die tatsächlich benötigte Bitanzahl (z. B. bei den Zahlen 0 oder 1), um sich dem vorgesehenen Zielvektor korrekt anzupassen.



Aufgabe: Typumwandlungen

Schreibe eine Zuweisung, die die Zahl 5 als 8-Bit-unsigned Vektor und die die Zahl –5 als 8-Bit-signed Vektor darstellt. Überprüfe in der Simulation die Binärdarstellung beider Vektoren.

Hinweis: Achte auf die Vektor-Länge im zweiten Parameter der Konvertierungsfunktion.

2.5 8-Bit-Addierer mit Carry-out

Als erstes praktisches Beispiel betrachten wir einen **8-Bit-Addierer**. Addierer sind grundlegende Bausteine der digitalen Technik und bilden die Basis für Zähler, Rechenwerke und viele weitere Schaltungen. Das folgende VHDL-Design zeigt eine mögliche Beschreibung.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Addierer8 is
```

```

port (
  A, B : in  std_ulogic_vector(7 downto 0);
  SUMME : out std_ulogic_vector(7 downto 0);
  COUT : out std_ulogic
);
end Addierer8;

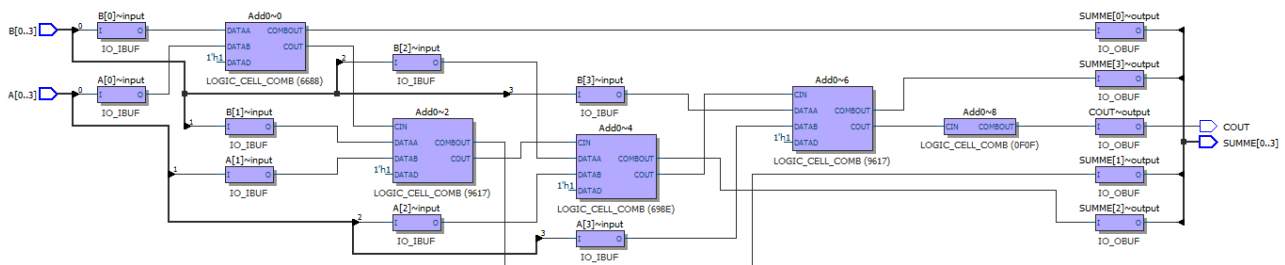
architecture Verhalten of Addierer8 is
  signal A_INT, B_INT, SUM_INT : unsigned(8 downto 0);
begin
  A_INT <= unsigned('0' & A); -- auf 9 Bit erweitern
  B_INT <= unsigned('0' & B);
  SUM_INT <= A_INT + B_INT;

  SUMME <= std_ulogic_vector(SUM_INT(7 downto 0)); -- niederwertige 8 Bit
  COUT <= SUM_INT(8); -- höchstwertiges Bit = Carry
end Verhalten;

```

Interessant ist, was der Compiler aus dieser VHDL-Beschreibung „baut“. Im RTL-Viewer erscheint der Addierer lediglich als ein einziger Block – was wenig aussagekräftig ist, da ein Addierer ein Grundbaustein der Register-Transfer-Ebene ist.

Spannender ist die **Technology Map**-Darstellung (Quartus: *Tools* → *Netlist Viewers* → *Technology Map Viewer*). Dort erkennt man, dass der Addierer aus vielen **Halbaddierern** und **Volladdierern** zusammengesetzt ist (z. B. Blöcke *Add0~0*, *Add0~2*). Mit *rechte Maustaste* → *Properties* lassen sich die Details einsehen.



Aufgabe: Simuliere den 8-Bit-Addierer mit verschiedenen Werten (z. B. $A = "11111111"$, $B = "00000001"$). Überprüfe, ob *COUT* korrekt gesetzt wird.

i Info: Veralterte Zusatzpakete

Früher wurden die **nicht IEEE-standardisierten** Pakete *std_logic_unsigned*, *std_logic_signed* und *std_logic_arith* oft verwendet. Damit konnte man direkt mit *std_logic_vector* rechnen – je nach eingebundenem Paket wurden **alle Vektoren** automatisch als *unsigned* oder *signed* interpretiert; *std_logic_arith* stammt von Synopsys und steht unter Copyright.

Weitere Probleme sind: Diese Pakete sind **nicht standardisiert** und werden von verschiedenen Herstellern unterschiedlich umgesetzt. Die automatische Interpretation aller Vektoren als *signed/unsigned* führt leicht zu Fehlinterpretationen und Fehlern. Best Practice ist daher die Verwendung von **numeric_std** mit den klar definierten Typen *unsigned* und *signed*.

 Aufgabe: Entwurf einer einfachen ALU

Erweitere den 8-Bit-Addierer („Addierer8“) zu einer einfachen ALU (Arithmetic Logic Unit). Lege dazu ein neues VHDL-Projekt mit dem Namen „ALU“ sowie der folgenden, vorgegebenen Schnittstelle an:

Eingänge:

A, B : std_ulogic_vector(7 downto 0)
op : std_ulogic_vector(3 downto 0)

Ausgänge:

Y : std_ulogic_vector(7 downto 0)
ZF : std_ulogic -- Zero Flag
CF : std_ulogic -- Carry Flag
SF : std_ulogic -- Sign Flag
OFL : std_ulogic -- Overflow Flag

Die ALU soll abhängig vom **Steuersignal „op“** unterschiedliche Operationen ausführen:

op = "0011" → Addition: Y = A + B
op = "0100" → NOT A: Y = not A
op = "0101" → SHL A: Y = A logisch um 1 Bit nach links verschoben

Hinweise:

- Beim SHL (Shift Left) wird das niederwertige Bit mit '0' aufgefüllt.
- Alle anderen op-Muster: Y <= A (keine Operation).

Flags:

- ZF = '1', wenn Y = 0, sonst '0'
- SF = höchstwertiges Bit von Y
- CF = Übertrag (Carry) bei der Addition
- OFL = arithmetischer Überlauf bei der Addition (signed)

Verwende zur Auswahl der Operation eine CASE-Anweisung und implementiere die ALU als kombinatorisches Schaltnetz.

3 Schaltwerke, Register & Taktteiler

Flip-Flops sind die elementaren Speicherelemente digitaler Systeme. Erst durch sie wird es möglich, **Zustände über die Zeit hinweg zu speichern**. Aus einer Zusammenschaltung von Flip-Flops entstehen wichtige Grundbausteine:

- **Register**, die z.B. Datenworte zwischenspeichern,
- **Zähler**, die z.B. Ereignisse zählen oder Adresssignale generieren
- **Taktteiler**, die z.B. ein langsames Taktsignal für einen Schaltungsteil erzeugen.

Diese Bausteine bilden die Grundlage für komplexere Schaltwerke wie endliche Automaten (FSMs) oder Prozessor-Kerne.

3.1 Wiederholung: Schaltwerke vs. Schaltnetze

Digitale Schaltungen lassen sich grundsätzlich in zwei Kategorien einteilen:

- **Schaltnetze**
 - Ausgänge hängen *nur* von den momentanen Eingangswerten ab.
 - Kein Gedächtnis, kein Speichern von Zuständen.
 - Beispiel: Addierer, Decoder, Multiplexer.
- **Schaltwerke**
 - Besitzen zusätzlich **Speicherelemente** (Flip-Flops, Register).
 - Ausgänge hängen sowohl von den aktuellen Eingängen **als auch vom vorherigen Zustand** ab.
 - Beispiel: Register, Zähler, Automaten.

 **Merke:** Ein Schaltwerk = Schaltnetz + Speicher. Es beschreibt damit ein **zeitabhängiges Verhalten**, da die Ausgänge durch Eingänge *und* vorherige Zustände bestimmt werden.

3.2 Register

Werden mehrere D-Flip-Flops parallel geschaltet, so entsteht ein **Register**. Jedes Flip-Flop besitzt dabei seinen eigenen Dateneingang und -ausgang; lediglich die gemeinsamen Steuerleitungen (Takt, Reset und ggf. Enable) werden von allen Flip-Flops gemeinsam genutzt. In VHDL beschreibt man solche Strukturen zweckmäßigerweise mit **Vektoren**, sodass mehrere Bits als ein Datenwort verarbeitet werden können.

Beispiel: 8-Bit-Register (positiv taktflankengesteuert)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Register8 is
    port (
        CLK : in  std_ulogic;
        D   : in  std_ulogic_vector(7 downto 0);
        Q   : out std_ulogic_vector(7 downto 0)
    );
end Register8;

architecture Verhalten of Register8 is
begin
    REG: process(CLK)
    begin
        if rising_edge(CLK) then
            Q <= D;    -- Eingangswert bei Taktflanke speichern
        end if;
    end process REG;
end Verhalten;
```

Beachte...

- Die Eingänge D bilden ein 8-Bit-Datenwort.
- Bei jeder steigenden Taktflanke (*rising_edge(CLK)*) übernimmt das Register den aktuellen Wert von D .
- Der gespeicherte Wert liegt bis zur nächsten Taktflanke am Ausgang Q an.
- Das Register verhält sich also wie ein **Speicher für ein Datenwort**, der synchron zum Takt aktualisiert wird.

 **Aufgabe:** Erweitere das Register auf **16 Bit**. Was ändert sich am VHDL-Code? Überprüfe im RTL-Viewer.

 **Aufgabe:** Erweitere das 8-Bit-Register so, dass es einen zusätzlichen **Enable-Eingang** besitzt. Das Register soll den Wert von D nur dann bei einer steigenden Taktflanke übernehmen, wenn $E = '1'$ ist. Ist $E = '0'$, bleibt der alte Wert am Ausgang Q erhalten. Überprüfe die Funktion in der Simulation, indem du verschiedene Szenarien testest (z. B. $E = '0'$ über mehrere Takte, Umschalten von E während des Takts).

Bisher haben wir verschiedene Registervarianten betrachtet – fest auf 8 Bit, auf 16 Bit, mit oder ohne Enable. Für jede Breite oder Variante eine eigene ENTITY zu schreiben, wäre jedoch umständlich und fehleranfällig. Eine elegante Lösung bietet VHDL mit dem Schlüsselwort **GENERIC**. Mit einem **GENERIC** lassen sich Parameterwerte definieren, die innerhalb einer ENTITY wie Konstanten behandelt werden. Damit können Hardwareelemente **variabel dimensioniert** werden, ohne den VHDL-Code für jede Variante neu schreiben zu müssen. Ein **GENERIC**-Wert kann zudem von einer höheren Hierarchieebene beim Instanzieren einer Komponente überschrieben werden.

Ein typisches Anwendungsbeispiel ist die einstellbare **Bitbreite eines Registers**:

```
entity Register is
  generic (N : integer := 8); -- Standardbreite = 8 Bit
  port (
    CLK, E      : in  std_ulogic;
    D           : in  std_ulogic_vector(N-1 downto 0);
    Q           : out std_ulogic_vector(N-1 downto 0)
  );
end Register;
```

Instanziierung für z.B. 16 Bit:

```
U1: entity work.Register
  generic map (N => 16)
  port map (
    CLK => CLK,
    E   => E,
    D   => D16,
    Q   => Q16
  );
```

... hier wird das Register **U1** mit einer Breite von 16 Bit instanziiert. Ohne Angabe eines **GENERIC MAP** würde automatisch der Standardwert $N=8$ verwendet.

Aufgabe: 7-Segment-Decoder mit generic ACTIVE_LOW

Überarbeite deine VHDL-Beschreibung des 7-Segment-Decoders aus Teil 1 so, dass die Ansteuerung über ein GENERIC **ACTIVE_LOW : boolean** gesteuert wird. Ist *ACTIVE_LOW = true*, soll der Decoder Ausgänge für eine Anzeige mit gemeinsamer **Anode** erzeugen (*Segmente aktiv bei '0'*; z.B. DE0-Board). Ist *ACTIVE_LOW = false*, soll der Decoder Ausgänge für eine Anzeige mit gemeinsamer **Kathode** erzeugen (*Segmente aktiv bei '1'*). Teste dein Design auf dem **DE0-Board**: Steuere die Anzeige *HEX3* korrekt mit *ACTIVE_LOW = true* an; die Anzeige *HEX0* jedoch mit *ACTIVE_LOW = false* (Segmente erscheinen „invertiert“).

3.3 Zähler

Ein **Zähler** ist eine spezielle Form eines Registers: Bei jeder Taktflanke wird der gespeicherte Wert nicht einfach überschrieben, sondern um **+1** (oder **-1**) verändert. Damit lassen sich Ereignisse mitzählen, Adressen durchlaufen oder Taktteiler aufbauen. Zähler sind daher einer der wichtigsten Grundbausteine digitaler Systeme.

Beispiel eines 4-bit Binärzählers:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Counter4 is
    port (
        CLK : in  std_ulogic;
        Q   : out std_ulogic_vector(3 downto 0)
    );
end Counter4;

architecture Verhalten of Counter4 is
    signal CNT : unsigned(3 downto 0);
begin
    PROC: process(CLK)
    begin
        if rising_edge(CLK) then
            CNT <= CNT + 1;
        end if;
    end process PROC;

    Q <= std_ulogic_vector(CNT);
end Verhalten;
```

Beachte...

- Das Signal *CNT* ist als **unsigned-Vektor** deklariert. Damit lassen sich Additionen direkt mit **+** ausführen.
- Bei jeder steigenden Taktflanke wird der Wert von *CNT* um **1** erhöht.
- Der aktuelle Wert wird am Ausgang *Q* als *std_ulogic_vector* ausgegeben.
- Nach **15 (IIII)** folgt wieder **0** – der Zähler läuft im **Modulus-16**.

Aufgabe: Zähler mit Reset

Erweitere den 4-Bit-Binärzähler um einen zusätzlichen **Reset-Eingang**. Bei *Reset* = '1' soll der Zählerstand sofort auf 0 gesetzt werden. Bei *Reset* = '0' soll der Zähler wie gewohnt bei jeder steigenden Taktflanke hochzählen. Simuliere den Zähler und überprüfe, dass das Zurücksetzen auf 0 unabhängig vom aktuellen Zählerstand korrekt funktioniert.

In VHDL können ganze **Arrays** (z. B. *std_ulogic_vector*, *unsigned*, *signed*) über **Aggregate** beschrieben werden. Das Schlüsselwort **OTHERS** steht dabei für „alle noch nicht explizit genannten Elemente“.

Vorteil von **OTHERS**:

- **Breitenunabhängig:** funktioniert für beliebige Vektorbreiten (auch wenn die Breite über *generic N* kommt).
- **Fehlerarm:** vermeidet Zählfehler bei „von Hand“ geschriebenen Bitmustern.
- **Lesbar:** sofort erkennbar als „alles auf 0/1/Z“.

Typische Einsatzfälle

- **Initialisierung / Reset auf Null**

```
signal CNT : unsigned(3 downto 0) := (others => '0'); -- Startwert in der
Simulation
...
if RESET = '1' then
    CNT <= (others => '0'); -- Reset in der Synthese
end if;
```

- **Einzelne Bits setzen, Rest löschen**

```
Q <= (7 => '1', 3 => '1', others => '0'); -- Bit 7 und 3 = 1, Rest = 0
Q <= (Q'high => '1', others => '0'); -- setzt nur das MSB
```

- **Mit Bereich oder Attribut (gleichwertig)**

```
Q <= (Q'range => '0'); -- gesamte Vektor-Range
Q <= (7 downto 0 => '0'); -- explizite Range
```

OTHERS deckt alle nicht genannten Elemente ab und darf nicht überlappen (z. B. dieselbe Stelle zweimal setzen). Ein Startwert in der **Deklaration** (*:= (others=>'0')*) wirkt sicher in der Simulation. Ob er in der Hardware wirklich als Power-Up-Wert umgesetzt wird, ist Tool/FPGA-abhängig. **Empfehlung:** Für deterministisches Verhalten immer **expliziten Reset** im Prozess codieren.

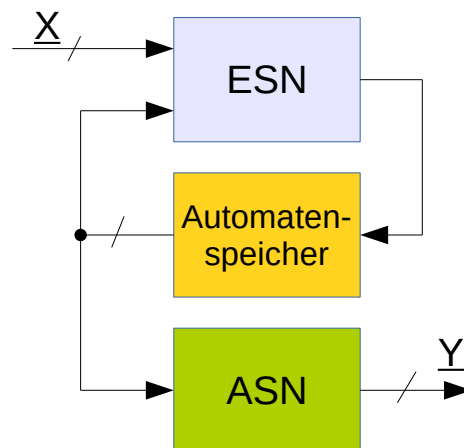
 **Merke:** (*others => '0'*) ist die **robuste Standardform** für Initialisierungen, Resets und „Rest füllen“ – unabhängig von der Vektorbreite.

Aufgabe: N-Bit-Zähler mit asynchronem Reset

Erweitere den 4-Bit-Zähler mit Reset zu einem **generischen N-Bit-Zähler**. Verwende ein *generic* ($N : \text{integer} := 8$) zur Definition der Bitbreite. Achte darauf, dass der Zähler bei $\text{Reset}='1'$ mit *others* zurückgesetzt wird. Simuliere dein Design mit unterschiedlichen Werten von N (z. B. 4, 8, 12) und überprüfe im RTL-Viewer, dass die Anzahl der Flip-Flops jeweils korrekt angepasst wird.

Ein Zähler lässt sich auch als **autonomer Automat** beschreiben – konkret als **Moore-Automat**:

- Der **Automatenspeicher (AS)** besteht aus einem getakteten Register.
- Das **Eingangsschaltnetz (ESN)** bildet die nächste Zustandsfunktion $\text{reg} + 1$.
- Das **Ausgangsschaltnetz (ASN)** liefert den aktuellen Registerwert nach außen.



```
architecture Automat of BinaryCounter is
    signal reg      : unsigned(3 downto 0); -- Automatenpeicher (akt. Zustand)
    signal reg_next : unsigned(3 downto 0); -- nächste Zustandsfunktion (ESN)
begin

    -- Automatenpeicher (AS)
    AS: process(CLK, RESET)
    begin
        if (RESET = '1') then
            reg <= (others => '0'); -- Initialzustand
        elsif rising_edge(CLK) then
            reg <= reg_next;       -- Zustandsupdate
        end if;
    end process AS;

    -- Eingangsschaltnetz (ESN)
    reg_next <= reg + 1;

    -- Ausgangsschaltnetz (ASN)
    Q <= std_ulogic_vector(reg);
end Automat;
```

 **Merke:** Ein Zähler ist ein Spezialfall eines Automaten:

Zustände = Zählerstände

Übergänge = Addition um +1

Ausgänge = aktueller Zustand

 Aufgabe: Dekadischer Zähler als Automat

Entwirf einen **dekadischen Zähler** (Zählumfang 0–9) in Form eines **Automaten**. Simuliere dein Design und überprüfe, dass der Zähler nach 9 wieder auf 0 zurückspringt.

Teste den Zähler auf dem **DE0-Board**: Verbinde die Zählerausgänge mit deinem 7-Segment-Decoder aus Teil 1. Überprüfe, dass die Ziffern 0–9 korrekt dargestellt werden.

 **KI-Aufgabe:** Frage eine KI, warum Zähler in VHDL-Designs häufig als **generische Bausteine** entworfen werden. Notiere die beiden wichtigsten Vorteile in eigenen Worten. Recherchiere zusätzlich, in welchen praktischen Anwendungen generische Zähler zum Einsatz kommen und vergleiche dies mit deinem N-Bit-Zähler.

3.4 Taktteiler

Digitale Systeme arbeiten **oft** mit einem **Taktsignal** als Zeitbasis. Dieses wird in der Praxis meist von einem **Quarzoszillator** bereitgestellt, da Quarze eine hohe Stabilität und Genauigkeit besitzen. Quarze sind jedoch nur in bestimmten Frequenzen verfügbar – nicht jede gewünschte Taktfrequenz lässt sich direkt realisieren.

In vielen Anwendungen muss ein vorhandener Takt daher auf eine **niedrigere Frequenz** heruntergeteilt werden. Auf dem DE0-Board steht beispielsweise ein Quarzoszillator mit **50 MHz** zur Verfügung. Für langsame Vorgänge, etwa das Blinken einer LED oder die Abtastung von Eingangssignalen, wird ein **Taktteiler** benötigt.

Ein Taktteiler wird in der Praxis fast immer durch einen **Zähler** realisiert.

- Der Zähler läuft mit der vollen Eingangsfrequenz.
- Abhängig vom aktuellen Zählwert wird ein **Ausgangssignal** abgeleitet – z. B. durch Verwendung eines höherwertigen Zählerbits oder durch einen Vergleich auf einen bestimmten Wert.
- Dadurch entsteht ein Teiltakt mit längerer Periodendauer und damit geringerer Frequenz.

Beispiel:

- Ein 4-Bit-Zähler läuft von 0 bis 15.
- Das höchstwertige Bit ändert sich nur alle 8 Takte → ergibt eine Frequenzteilung durch 16.

 **Merke:** Jeder Taktteiler ist im Kern ein Zähler, bei dem bestimmte Zählerstände oder Bits als „langsamer Takt“ verwendet werden.

Beispiel: Taktteiler durch 16

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ClockDivider16 is
  port (
    CLK    : in  std_ulogic;
    DIV16  : out std_ulogic
  );
end ClockDivider16;

architecture Verhalten of ClockDivider16 is
  signal CNT : unsigned(3 downto 0) := (others => '0');
begin
  PROC: process(CLK)
  begin
    if rising_edge(CLK) then
      CNT <= CNT + 1;
    end if;
  end process PROC;

  DIV16 <= CNT(3); -- Teiltakt: Frequenz = CLK / 16
end Verhalten;
```

Beachte...

- *CNT* ist ein 4-Bit-Zähler, der mit der steigenden Flanke von *CLK* hochzählt.
- Das Bit *CNT(3)* kippt bei jeder achten Taktflanke um → insgesamt ergibt sich eine **Teilung durch 16**.
- Allgemein gilt: Das Bit *CNT(n)* liefert einen Teiltakt mit der Frequenz $CLK / 2^{(n+1)}$.

In der Praxis werden jedoch oft **Teilerfaktoren** benötigt, die **keine Zweierpotenzen sind** – z. B. 10, 1000 oder 1 000 000. Auch in diesem Fall lässt sich ein Taktteiler mit einem Zähler realisieren: Der Zähler läuft bis zu einem definierten Maximalwert und wird dann zurückgesetzt, während gleichzeitig der Ausgang toggelt.

Beispiel: Taktteiler durch 20

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity ClockDivider20 is
  port (
    CLK    : in  std_ulogic;
    DIV10  : out std_ulogic
  );
end ClockDivider20;
```



```
architecture Verhalten of ClockDivider20 is
    signal CNT : unsigned(3 downto 0) := (others => '0'); -- Zähler 0..9
    signal TOG : std_ulogic := '0';
begin
    PROC: process(CLK)
    begin
        if rising_edge(CLK) then
            if CNT = 9 then
                CNT <= (others => '0'); -- zurücksetzen
                TOG <= not TOG;         -- Ausgang toggeln
            else
                CNT <= CNT + 1;
            end if;
        end if;
    end process PROC;

    DIV10 <= TOG; -- Teiltakt = CLK / 20
end Verhalten;
```

Beachte...

- CNT läuft von 0 bis 9 → danach Reset auf 0.
- Jedes Mal, wenn CNT=9 erreicht ist, wird das Ausgangssignal TOG invertiert.
- Damit ergibt sich:
TOG wechselt alle 10 Takte,
also Periodendauer = 20 Eingangszyklen,
Frequenz = $CLK / 20$.

 Aufgabe: Taktteiler 50 MHz → 1 Hz

Erstelle einen Taktteiler, der den Systemtakt des DE0-Boards (50 MHz) auf 1 Hz herunterteilt. Verwende dazu einen geeigneten **Zähler**, der bei Erreichen eines Maximalwerts zurückgesetzt wird und ein Ausgangssignal toggelt.

Berechne vorab, wie groß der Zähler sein muss, um aus 50 MHz eine Periodendauer von 1 s zu erhalten. Überprüfe die korrekte Funktion mittels Simulation. Teste Deine VHDL-Beschreibung, indem Du das resultierende 1 Hz-Signal auf einer **LED** am Board ausgibst und überprüfe, dass sie im Sekundentakt blinkt.

 **KI-Aufgabe:** Frage eine KI, wie man eine gewünschte Ausgangsfrequenz erzeugen kann, wenn sich der Teilerfaktor nicht als ganze Zahl darstellen lässt (z. B. aus 50 MHz → 3 MHz). Recherchiere mindestens **zwei verschiedene Verfahren**, die dafür in der Praxis eingesetzt werden.

Entwickle zusammen mit der KI eine **VHDL-Beschreibung eines Taktteilers**, der aus 50 MHz ein 3 MHz-Signal erzeugt. Verwende dabei ausschließlich Sprachkonstrukte, die wir bisher behandelt haben. **IP-Komponenten sind nicht erlaubt.**

Das Teilen eines Taktsignals über Flip-Flops benötigt viele Logikressourcen (LEs). Dafür stehen im Cyclone III vier interne **Phase-Locked Loops (PLLs)** zur Verfügung. Eine PLL ist eine

phasengekoppelte Regelungsschleife, die eine Eingangstaktfrequenz intern vervielfacht oder teilt und so neue, phasenstabile Taktsignale bereitstellt. Im FPGA ist die PLL als dedizierter Hard-IP-Block realisiert und erlaubt eine deutlich effizientere Takterzeugung als eine reine Flip-Flop-Teilerkette. Da nur bestimmte Teiler- und Multiplikationsfaktoren unterstützt werden, wird in der Praxis häufig eine Kombination aus PLL und nachgeschaltetem Zähler eingesetzt.

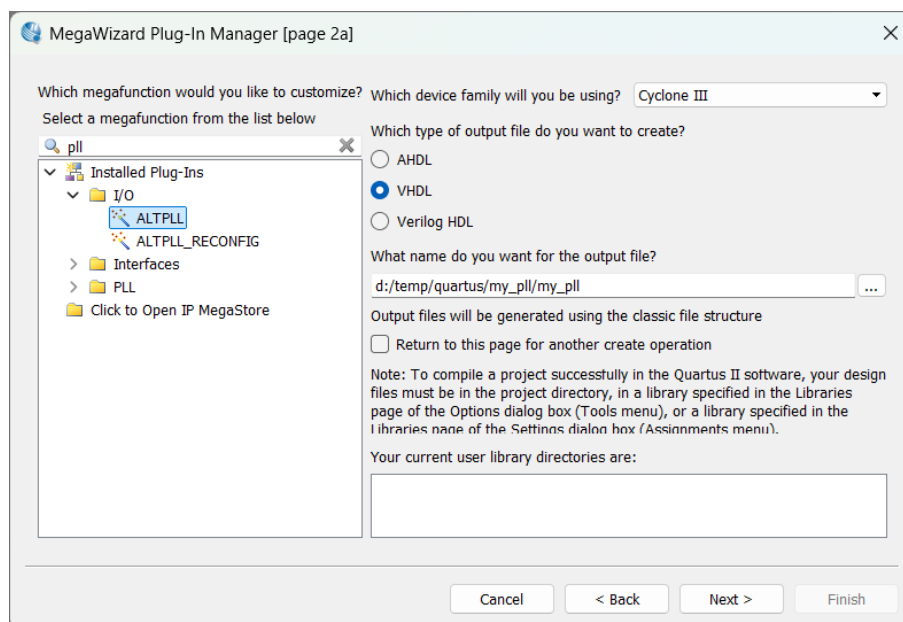
Beispiel: 27 MHz Takt

Auf dem DE0-Board befindet sich ein Quarzoszillator mit 50 MHz. Für eine Anwendung wird jedoch ein Takt mit 27 MHz und 50 % Tastverhältnis benötigt. Dieses soll nun mithilfe der PLL aus dem 50 MHz-Signal erzeugt werden.

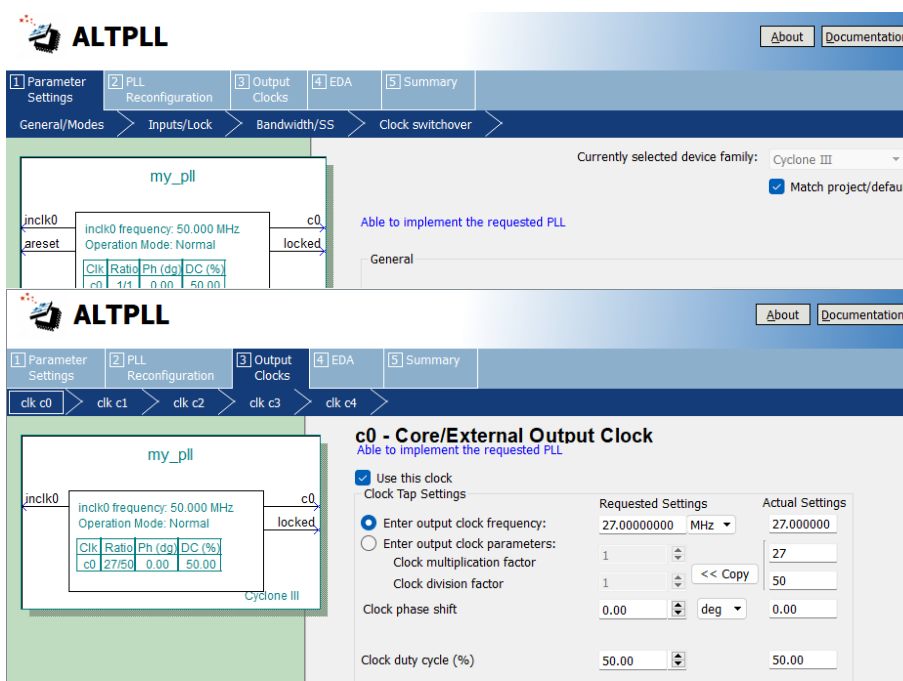
1. MegaWizard öffnen:

Tools → MegaWizard Plug-In Manager → Create a new custom megafunction variation

2. **ALTPLL** auswählen und eine neue VHDL-Komponente (z. B. *my_pll*) erstellen.



3. Eingangsfrequenz (50 MHz) angeben, bei *Output Clocks* die gewünschte Ausgangsfrequenz (27 MHz) konfigurieren.



- Die Standardoptionen übernehmen und mit *Finish* abschließen. Quartus bietet an, die Komponente direkt ins Projekt einzubinden.

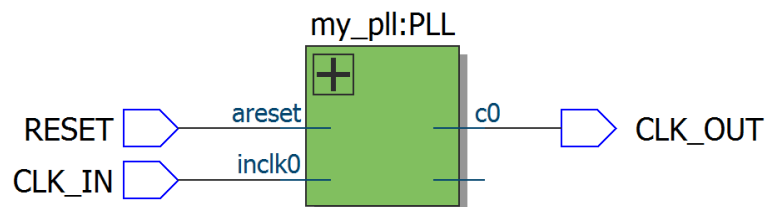
Einbindung der *my_PLL*-Komponente in eine VHDL-Beschreibung:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity PLL_DEMO is
  port (
    RESET, CLK_IN : in  std_ulogic;
    CLK_OUT       : out std_ulogic
  );
end PLL_DEMO;

architecture Einsatz_PLL of PLL_DEMO is
begin
  -- Instanzierung der PLL-Komponente
  PLL: entity work.my_pll
    port map (
      areset => RESET,
      inclk0 => CLK_IN,
      c0     => CLK_OUT,
      locked => open  -- Locked-Ausgang bleibt hier ungenutzt
    );
end Einsatz_PLL;
```

RTL-Viewer:



Aufgabe: Taktteiler mit PLL und FF-Kette

Erzeuge aus dem **50 MHz-Systemtakt** des DE0-Boards ein **1 Hz-Signal**. Verwende diesmal eine **PLL** zur Voranpassung der Frequenz und anschließend einen **Zähler (FF-Kette)** zur weiteren Teilung. Vergleiche das Design mit deiner vorherigen Lösung, die ausschließlich eine Zählerkette verwendet hat. Ermittle im Synthesereport die **Anzahl der belegten Logikelemente (LEs)** in beiden Fällen. Welcher Ansatz ist ressourcenschonender und damit für den praktischen Einsatz im FPGA vorzuziehen?

4 Simulation mit ModelSim

Bisher haben wir die integrierte Waveform-Simulation von Quartus verwendet. Für

komplexere Designs bietet sie jedoch nicht genug Flexibilität. In der Praxis ist **ModelSim** (heute oft auch *QuestaSim*) die Standardumgebung für VHDL-Simulationen. ModelSim erlaubt die Erstellung von **Testbenches**, das gezielte Beobachten von Signalen und das systematische Verifizieren des Designs.

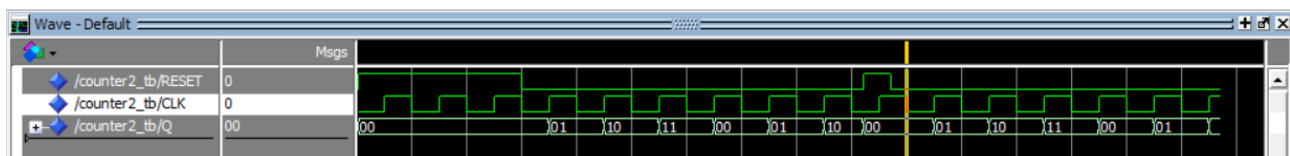
Als Beispiel verwenden wir einen **2-Bit-Zähler mit asynchronem Reset**, der durch Instanziierung des zuvor entwickelten N-Bit-Zählers mit asynchronem Reset realisiert wird.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Counter2 is
  port (
    CLK    : in  std_ulogic;
    RESET  : in  std_ulogic;
    Q      : out std_ulogic_vector(1 downto 0)
  );
end Counter2;

architecture Struktur of Counter2 is
begin
  CNT2: entity work.CounterN
    generic map (N => 2)          -- 2-Bit-Zähler
    port map (
      CLK    => CLK,
      RESET  => RESET,
      Q      => Q
    );
end Struktur;
```

Simulationsergebnis mit ModelSim:



Für die Simulation in ModelSim wird eine sogenannte **Testbench** benötigt. Eine Testbench ist eine VHDL-Beschreibung, die **keine Ports** besitzt. Sie erzeugt intern alle Testsignale und verbindet diese per **hierarchischem Design** mit dem zu testenden Modul – meist als *DUT* (*Device Under Test*) bezeichnet.

Einige wichtige Unterschiede und Besonderheiten:

- **Startwerte für Signale:** In einer Testbench können Signale direkt mit Anfangswerten versehen werden (z. B. `signal clk : std_ulogic := '0';`). Das dient ausschließlich der Simulation und hat keine Auswirkung auf die Synthese.
- **Signalreihenfolge:** Die Reihenfolge der Signaldeklarationen bestimmt die Reihenfolge der Signaldarstellung im Wavefenster von ModelSim.
- **Zeitsteuerung:** Mit `wait for ...` lassen sich Zeitverzögerungen definieren, um Taktzyklen oder Stimuli präzise zu steuern.

- **Taktgenerator:** Ein Taktsignal wird in einem **eigenen Endlosprozess ohne Sensitivitätsliste** erzeugt. Dieser Prozess toggelt das Signal regelmäßig.
- **Stimuli:** Die eigentlichen Testeingaben (z. B. Reset-Pulse, Eingangswerte) werden in einem separaten Prozess formuliert. Da in Prozessen die Anweisungen sequentiell abgearbeitet werden, kann man hier klar strukturierte Testabläufe beschreiben.

 **Merke:** Eine Testbench beschreibt keine Hardware, sondern dient ausschließlich der Simulation. Sie darf deshalb auch Konstrukte enthalten, die für die Synthese nicht erlaubt oder sinnvoll sind.

Beispiel: Einfache Testbench für den 2-Bit-Zähler

```
-- Counter2_tb.vhd
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Counter2_tb is
end entity;

architecture TB of Counter2_tb is
    -- Testbench-Signale in der Reihenfolge der späteren Darstellung
    signal RESET : std_ulogic := '0';
    signal CLK   : std_ulogic := '0';
    signal Q     : std_ulogic_vector(1 downto 0);

begin
    -- Device Under Test (DUT)
    DUT: entity work.Counter2
        port map (
            CLK   => CLK,
            RESET => RESET,
            Q     => Q
        );

    -----
    -- Taktgenerator: eigener Prozess, keine Sensitivitätsliste
    -----
    CLK_GEN : process -- Periodendauer: 100ns (= 10 MHz; als Beispiel)
    begin
        CLK <= '0';
        wait for 50ns;
        CLK <= '1';
        wait for 50ns;
    end process CLK_GEN;

    -----
    -- Stimulus: Reset-Sequenzen und Beobachtung des Zählerverhaltens
    -----
    STIM : process
    begin
        -- Test 1: asynchroner Reset aktiv für 3 Taktzyklen
        RESET <= '1';
```

```

wait for 3*100ns;

-- Test 2: Reset inaktiv, normal zählen lassen
RESET <= '0';
wait for 6*100ns;

-- Test 3: asynchroner Reset-Puls *zwischen* zwei Flanken
wait for 20ns;    -- bewusst asynchron zur Taktflanke
RESET <= '1';
wait for 50ns;
RESET <= '0';
wait for 600ns;

-- Simulation ordentlich beenden
assert false report "Simulation completed" severity failure;
end process STIM;

end architecture TB;

```

i Info: Warum Änderungen in der Testbench sofort sichtbar sind

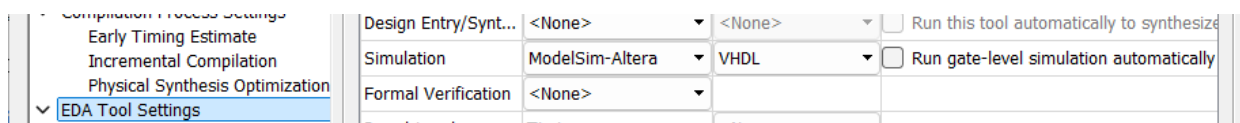
Signalzuweisungen in VHDL werden grundsätzlich vorgemerkt und erst nach Ende des Prozesses wirksam. In einer Testbench enthalten die Prozesse jedoch *WAIT*-Anweisungen, die den Ablauf unterbrechen. Nach jedem *WAIT* gilt der Prozess formal als beendet, und die zuvor geplanten Signaländerungen werden sofort umgesetzt.

Dadurch erscheinen Änderungen in der Testbench „sofort sichtbar“, obwohl weiterhin das normale VHDL-Simulationsmodell gilt.

Einbinden der Testbench in Quartus und Start der ModelSim-Simulation

1. EDA-Tool einstellen:

Assignments → *Settings* → *EDA Tool Settings*; *Simulation*: *ModelSim-Altera*, *VHDL*.



2. Testbench anlegen:

Entweder per Template Writer (*Processing* → *Start* → *Start Testbench Template Writer*; erzeugt .vht) **oder** direkt selbst schreiben (empfohlen).

3. Testbench ins Projekt aufnehmen:

Project Navigator → *Add/Remove Files...* und die Testbench dem Projekt hinzufügen.

4. Testbench in den Projekteinstellungen einbinden:

Assignment → *Settings* → *EDA Tool Settings* → *Simulation* → *Compile test bench ...*

Create new test bench settings.

Test bench name:

Top level module in test bench:

☐ Use test bench to perform VHDL timing simulation

Design instance name in test bench:

Simulation period

☒ Run simulation until all vector stimuli are used

☐ End simulation at: s

Test bench and simulation files

File name: ...

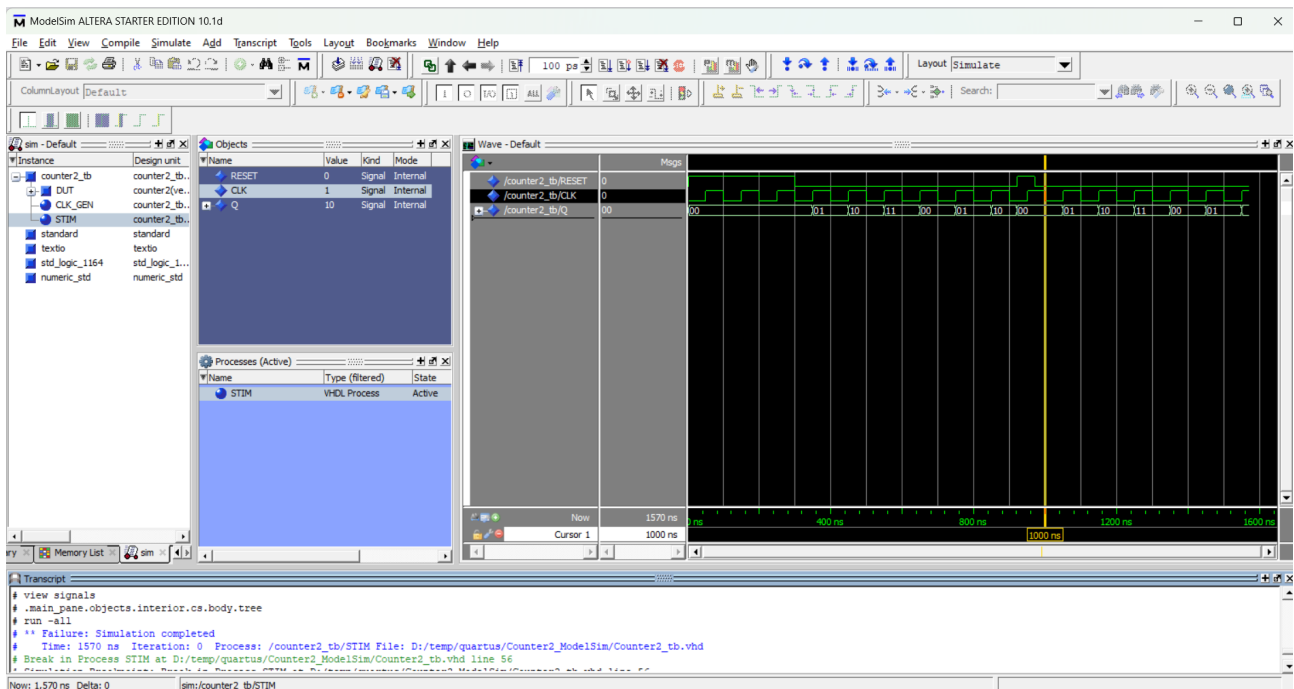
File Name	Library	HDL Version
Counter2_tb.vhd		Default

Buttons: Add, Remove, Up, Down, Properties...

Buttons: OK, Cancel, Help

5. RTL Simulation starten:

Tools → *Run Simulation Tool* → *RTL Simulation*



Aufgabe: Simulation des Parity-Checkers mit ModelSim

Verwende deine VHDL-Beschreibung des **Parity-Checkers** (Kapitel 1). Erstelle eine passende **Testbench**:

- Deklariere ein Signal *D* (Vektor) und das Ausgangssignal *Gerade*.
- Lege verschiedene Testwerte für *D* fest (z. B. "0000", "0001", "1111").
- Verwende *WAIT FOR* ..., um die Eingaben zeitlich zu staffeln.

Simuliere das Design in ModelSim und überprüfe, dass der Ausgang *Gerade* genau dann '1' wird, wenn die Anzahl der Einsen gerade ist.

5 Projekt: Stoppuhr

Als Abschlussprojekt soll eine **Stoppuhr** entworfen werden. Dieses Projekt integriert alle bisher vermittelten Inhalte:

- **Zähler und Taktteiler** zur Erzeugung der Zeitbasis,
- **Register** zur Speicherung der Zählerstände,
- **Eingabesteuerung** (Buttons) für Start, Stop und Reset,
- **Dekoder** zur Ansteuerung der 7-Segment-Anzeigen,
- **hierarchisches Design** zur Strukturierung des Projekts.

Funktionsbeschreibung:

- Mit *Button1* wird die Uhr gestartet.
- Mit *Button2* wird die Uhr angehalten.

- Mit *Button0* wird die Anzeige auf 00.00 zurückgesetzt.
- Die Ausgabe erfolgt auf den vier 7-Segment-Anzeigen: zwei Stellen für Sekunden sowie Zehntel- und Hundertstel-Sekunden.

Vorgabe (Top-Level-Entity):

```
entity Stopwatch is
  port(
    -- 50 MHz Systemtakt
    CLOCK_50 :    in std_ulogic;

    -- Buttons für Start / Stop / Reset
    BUTTON    :    in std_ulogic_vector(3 downto 0);

    -- 7-Segment-Anzeigen...
    HEX0_D    :    out std_ulogic_vector(6 downto 0); -- gfedcba
    HEX0_DP   :    out std_ulogic;
    HEX1_D    :    out std_ulogic_vector(6 downto 0);
    HEX1_DP   :    out std_ulogic;
    HEX2_D    :    out std_ulogic_vector(6 downto 0);
    HEX2_DP   :    out std_ulogic;
    HEX3_D    :    out std_ulogic_vector(6 downto 0);
    HEX3_DP   :    out std_ulogic
  );
end Stopwatch;
```

Die Pinzuordnung erfolgt über die vorbereitete Datei *Pin_DE0_TOP.csv*:

Assignments → *Import Assignments* → *Auswahl Pin_DE0_TOP.csv* → *OK*

Hinweis zur Blockstruktur

Die Stoppuhr sollte modular aufgebaut werden:

- **PLL / Taktteiler**: Herunterteilen des 50-MHz-Quarzoszillators auf eine Zeitbasis.
- **Automat** für Start/Stop/Reset-Steuerung.
- **Zähler** für Hundertstel, Zehntel und Sekunden.
- **7-Segment-Dekoder** für die Ausgabe.

Wichtig: Ein komplexes Projekt wird beherrschbar, wenn es in **Teilblöcke** zerlegt und hierarchisch verbunden wird.

Mögliche TopLevel-Struktur:

